Parallel Dynamic Partitioning for Datapath Combinational Equivalence Checking

Shuai Zhou*§, Weikang Zhang*, Xindi Zhang†, Zite Jiang*§, Haihang You*¶, Shaowei Cai†¶
*Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
†Key Laboratory of System Software (Chinese Academy of Sciences) and
State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing, China
§School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing, China
Emails: {zhoushuai22s, zhangweikang, jiangzite19s}@ict.ac.cn; zhangxd@ios.ac.cn; caisw@ios.ac.cn; youhaihang@ict.ac.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132837

***Abstract*—Combinational Equivalence Checking (CEC) is a crucial technique in electronic design automation for verifying the functional equivalence of combinational circuits. Recently, combinational circuit design increasingly incorporates more complex arithmetic structures, commonly known as datapath circuits. However, existing state-of-the-art tools often exhibit subpar performance in solving datapath CEC problems. To further advance the exploration on datapath CEC process, this study introduces PDP-CEC (Parallel Dynamic Partitioning Combinational Equivalence Checking), a novel parallel CEC approach integrating circuit partitioning and dynamic task scheduling into the CEC process, enhancing the efficiency of CEC for datapath circuits. PDP-CEC introduces an innovative method for selecting critical nodes to split the search space of the CEC problem, facilitating the efficient generation of numerous independent subproblems. Meanwhile, a dynamic task scheduling strategy is implemented in PDP-CEC to ensure load balancing and prevent hard-to-solve subproblems from stalling the entire process. Compared to the most advanced tools such as ABC and Hybrid-CEC, PDP-CEC significantly accelerates CEC process, achieving speedups ranging from 5.11x to 125.27x, while effectively solving approximately three times more datapath CEC problems. With excellent scalability, PDP-CEC shows substantial improvements in combinational equivalence checking for datapath circuits, offering an efficient parallel approach to meet the demands of large-scale datapath CEC tasks.**

***Index Terms*—Combinational Equivalence Checking, parallel computing, circuit partitioning, dynamic task scheduling.**

I. Introduction

Combinational equivalence checking (CEC) involves verifying whether two combinational circuits are functionally equivalent. In contemporary circuit design processes [1], circuits are optimized through structural rewriting [2] or logic synthesis [3] after the RTL design stage. CEC is then employed to ensure that the circuit's functionality remains consistent before and after those optimizations. Its further applications cover various scenarios in electronic design automation (EDA) and digital integrated circuit design, including functional equivalence removal [4], computation of structural choice [5] and sequential equivalence checking [6].

Today, there has been a clear trend toward designing combinational circuits with increasingly complex arithmetic components [7]. Modern circuits are typically composed of fundamental arithmetic units, such as multipliers and multiplexers, collectively referred to as datapath circuits [8]. However, state-of-the-art CEC tools that are based on the SAT sweeping algorithm [9] [10] perform poorly when applied to datapath circuits.

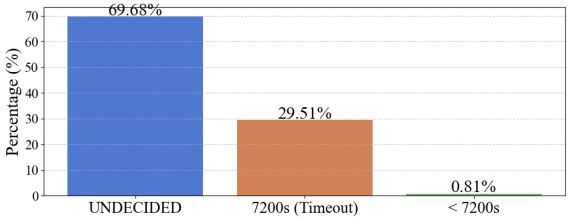

Fig. 1. Result distribution of ABC "&cec" on 742 industrial datapath circuits. "UNDECIDED" indicates the tool's inability to generate the result.

For example, 742 industrial datapath circuit cases were verified with the ABC "&cec" command [11], a widely used tool for logic synthesis and formal verification in both industry and academia, whose core is the SAT sweeping algorithm. As shown in Fig. 1, nearly 70% of the cases returned "UNDECIDED", while 29% produced no results after two hours. Only 1% completed verification within the time limit.

To improve the weak performance of these tools on datapath CEC problems, prior studies have predominantly concentrated on optimizing the CEC solving process, including more intelligent simulation methods [9] and unified representations [12]. As research on serial algorithms reaches saturation, it becomes crucial to explore universal parallel workflow for datapath circuits. There are two key challenges in parallelizing the datapath CEC process:

- **Ineffective partition** Circuits with high coupling and low locality, if divided naively, risk disrupting functional dependencies, which will create interdependent subcircuits. To generate independent subcircuits, the existing parallel approach [13] partitions circuits with Transitive Fan-In (TFI) cones of Primary Outputs (POs). Similarly, RepCut [14] employs hypergraph partition based on TFI levels. However, these partitioning methods that rely on TFI do not be well-suited for datapath circuits, which typically have a limited number of POs.

¶ Corresponding author

- **Load imbalance** Due to the inherent unpredictability of each subcircuit's workload, driven by the SAT solver at the core of CEC, efficiently balancing tasks in parallel execution becomes challenging. A parallel strategy [15] has been proposed in Satisfiability Modulo Theories (SMT) domain, with a binary tree to store subproblems' results and guide thread actions. However, this approach falls short for the parallel CEC workflow. Starting the execution from the root node's task can lead to significant thread stalling, particularly for the root problem and its nearby subproblems.

To overcome these challenges, we propose PDP-CEC (Parallel Dynamic Partitioning Combinational Equivalence Checking), a novel parallel CEC approach for datapath circuits that integrates circuit partitioning and dynamic task scheduling into the CEC process.

The circuit partitioning algorithm in PDP-CEC utilizes a distinctive node selection method to split the search space, effectively bypassing structural constraints and generating numerous independent subproblems. It incorporates a heuristic to estimate the complexity reduction achieved through partitioning, ensuring that removing assignment nodes propagated and inferred from the selected nodes substantially diminishes the circuit's overall complexity. To enhance this process, we integrate the Metis graph partitioning algorithm [16] and develop an approach for selecting suitable nodes in alignment with the circuit's structure.

Furthermore, PDP-CEC implements a recursive workflow based on a binary splitting tree to guide task scheduling during execution, which shows in Fig. 2. PDP-CEC employs the "master-worker" structure, comprising a master for expanding the binary splitting tree, a monitor for managing task scheduling, and workers for solving subproblems in parallel. With master pre-partitioning the circuit to initialize the tree, the task scheduling strategy helps PDP-CEC attain load balancing among workers, effectively reducing resource waste from idle workers and preventing hard-to-solve subproblems from stalling the entire process.

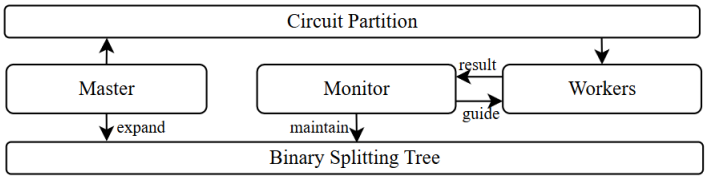

Fig. 2. The thumbnail of PDP-CEC's workflow.

In summary, we present PDP-CEC to fully exploit the potential of circuit structures, and our work makes the following contributions:

- We propose PDP-CEC, which integrates circuit partitioning and dynamic task scheduling to parallelize datapath CEC. With excellent scalability, PDP-CEC considerably outperforms state-of-the-art tools like ABC and Hybrid-CEC achieving speedups ranging from 5.11x to 125.27x with 128 threads in solving time and effectively resolving nearly three times more datapath CEC problems.
- We design a circuit partitioning algorithm for PDP-CEC that employs targeted node selection to split the search space, bypassing structural constraints and generating independent subproblems while ensuring that removing assignment nodes propagated and inferred from the selected nodes substantially decreases the circuit's overall complexity.
- We introduce a dynamic task scheduling strategy to attain load balancing during execution, avoiding resource waste from idle workers and preventing hard-to-solve subproblems from stalling the entire process.

Paper Organization Section II introduces the SAT sweeping algorithm. Section III presents the circuit partitioning algorithm and heuristic for selecting crucial nodes. Section IV details the workflow of PDP-CEC and gives an example of our dynamic scheduling strategy. Section V shows the experimental results, comparing PDP-CEC with state-of-the-art CEC tools. Finally, Section VI concludes the study and discusses future work.

II. Preliminaries

In this section, we introduce the typical CEC process which relies on the *SAT sweeping* algorithm and give an overview in Fig. 3.

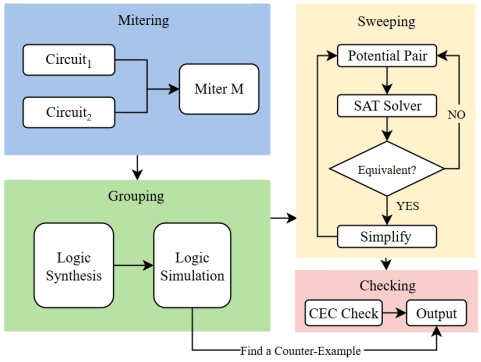

Fig. 3. The typical CEC process with SAT sweeping algorithm.

A complete CEC process typically involves four stages: constructing a miter circuit through mitering, grouping functionally potential-equivalent nodes by logic synthesis [17] and simulation, employing a SAT solver to identify potential-equivalent nodes and simplify the original miter circuit in a topological order through sweeping, and finally, checking the equivalence of the simplified circuit as the output.

Primarily suited for datapath CEC problems, Hybrid-CEC process [18] differs from traditional CEC in two key aspects: (1) it employs the ISD method [18] to reduce the number of pairs requiring verification by comparing TFI's structure with previously proven equivalent pairs, and (2) it partly replaces the SAT solver with the Exact Probability-based Simulation (EPS) method [19], which is better suited for solving low-

width pairs in datapath circuits. However, this method cannot serve as a comprehensive solution for all datapath circuits.

III. Circuit Partitioning Algorithm

Conventional graph partitioning methods often fail to yield parallelizable subcircuits because of high coupling within datapath circuits. To confront the structural intricacies of these circuits, we employ search space splitting to partition them into subcircuits for the following two main reasons:

- Structural information is lost when circuits are sent to the SAT solver. Search space partitioning on circuit level allows for better use of this information.
- Search space splitting avoids the intricate connectivity between gate nodes, focusing on dividing the search space instead of the structure.

In an And-Inverter Graph (AIG), each node represents an AND gate (or an XOR gate after preprocessing in PDP-CEC) with a maximum fan-in of 2 and a minimum fan-out of 1, except for output gates. Given the inherent characteristics of the circuit, each gate can only take values of 0 or 1, corresponding to low or high voltage levels. When assigning 0 or 1 to a gate, the search space of the original circuit is effectively divided into two independent regions. Based on the assignment, forward inference along the input direction and backward propagation along the output direction will be performed to assign new nodes through the property of gates. Removing those gates with assignments modifies the circuit structure, resulting in two smaller, independent subcircuits that each cover half the search space of the original circuit. We refer to this process of partitioning through assignment propagation as circuit incremental partitioning.

Algorithm 1 IncrementalPartitioning

1: **Input:** $TreeLevel\ L, Miter\ M(AIG)$
2: **Output:** $BinaryTree\ BStree$
3: Insert M into $BStree$
4: $Queue\ BSque$
5: $BSque.push(M)$
6: **while** $BStree.level \leq L$ **do**
7: $u \leftarrow BSque.pop()$
8: $x \leftarrow$ NodeSelecting(u)
9: $C_0 \leftarrow$ AssignValue$(u, x, 0)$
10: $C_1 \leftarrow$ AssignValue$(u, x, 1)$
11: Insert C_0 and C_1 into $BStree$
12: Push C_0 and C_1 into $BSque$
13: **end while**
14: **return** $BStree$

The Algorithm 1 describes the process of generating a binary splitting tree $BStree$ with a specified level L from the input miter circuit M. As the tree unfolds layer by layer, each parent node branches into two child nodes, representing an equal partitioning of the problem space. This partitioning process continues until it reaches the setting level L, which is determined by the circuit complexity and the number of threads available for parallel execution.

A. Subcircuits Creation

In lines 9 and 10 of Algorithm 1, the *AssignValue* function is invoked to partition circuit $C(V, E)$ into two subcircuits, $C_0(V_0, E_0)$ and $C_1(V_1, E_1)$ by assigning the gate x to 0 and 1, respectively. In the circuit topology, signals assigned to one gate can propagate in two directions. The term *inference* describes backward propagation along the input direction; for instance, if an AND gate's output is 1, both inputs must also be 1. Conversely, forward propagation occurs along the output direction, referred to as *propagation*. In this case, an AND gate's output of 0 will cause the output of the subsequent gate to be 0 as well, unless an inverter is present between them. Removing the assigned gates divides the circuit into two subcircuits with a combined search space equivalent to that of the original. In an AIG, this removal may create single-input gates, which can be simplified to wires, effectively reducing the overall circuit complexity.

Uncertain gates are stored as constraints. For instance, if an AND gate is assigned a 0, the constraint is that both inputs cannot simultaneously be 1, represented as (x_1,x_2,TT). These constraints serve as unique properties of the subcircuits and are subsequently converted into CNF expressions for SAT solver.

B. Nodes Selection

In line 8 of Algorithm 1, the *NodeSelecting* function is called to select appropriate nodes for first assignment. The selection of nodes is critical for the efficiency of the parallel execution process, as we aim to always choose nodes that significantly reduce circuit complexity during partitioning. Before introducing the algorithm for node selection, we first present the heuristic formula used to estimate the complexity of datapath circuit for SAT solving, which is evaluated by counting the number of XOR and AND gates, as expressed in the following formula:

$$\text{Complexity} = 3\log_2(\textit{num}_x) + \log_2(\textit{num}_a), \tag{1}$$

where $\textit{num}_x$ and $\textit{num}_a$ represent the number of XOR and AND gates, respectively.

In datapath circuits, components like multipliers and multiplexers are primarily built from adders, where XOR gates calculate sums and AND gates generate carries. Thus, the number of XOR gates, as a key indicator of the number of adders, carries more weight in (1) while AND gate count reflects circuit size. After partitioning, the number of removed nodes helps estimate subcircuit complexity using (1). However, calculating the exact impact of each node is time-consuming, so we further propose the following heuristic formulas to estimate the extent to which the selected nodes reduce overall circuit complexity:

$$\text{NC} = \frac{\textit{edge}_b - \textit{edge}_b \cdot \left(p_t \cdot \alpha \cdot (1 - \textit{rate}_x)\right)^{\textit{level}}}{1.0 - p_t \cdot \alpha \cdot (1 - \textit{rate}_x)}, \tag{2}$$

$$\text{Eval} = 3\log_2(\text{NC} \cdot \textit{rate}_x) + \log_2(\text{NC} \cdot (1 - \textit{rate}_x)). \tag{3}$$

The heuristic formula (2) for estimating the reduction in node count (NC) after partitioning is based on the geometric

series sum formula. The first term, $edge_b$, represents the number of different types of output edges, as edges with opposite signal assignments can aid propagation. The common ratio is $p_t \cdot \alpha \cdot (1 - rate_x)$, where p_t is the probability of a positive output edge, α is the average number of outgoing edges from a node, and $rate_x$ is the probability of XOR gates in the circuit. The variable *level*, representing the node's distance to the circuit's PO, helps gauge the propagation range. The 0 signal will continue to propagate through positive output edges after one round of propagation, so we use the probability of a positive output edge in the common ratio. By splitting NC into AND and XOR gates with $rate_x$, and applying (1), we can calculate the complexity reduction (Eval) for each node in (3). The node with the highest evaluation score (Eval) is chosen as the initial assignment node for partitioning, whose details are described in Algorithm 2.

Algorithm 2 NodeSelecting

```
Input: Circuit C
Output: Node N
Array Nodes ← GetPotentialNodes(C)
for node ∈ Nodes do
    h0 ← EvalCompute(node, 0)
    h1 ← EvalCompute(node, 1)
    node.sum_h ← h0 + h1
end for
best_node ← max_sum_h(Nodes)
return best_node
```

Algorithm 2 first identifies the range of potential optimal nodes in the circuit. For circuits solved with EPS, the range of selection is limited to the primary inputs (PI), as the number of PIs is a key factor influencing the speed of EPS. Since XOR chains pose a challenge for SAT-based methods [18], the Metis algorithm is applied to identify a cut set containing most of the XOR gates, aiming to reduce the number of XOR gates through incremental partitioning for circuits solved with SAT. The node that yields the greatest complexity reduction is then selected as the initial assignment node.

IV. Dynamic Task Scheduling Workflow

In PDP-CEC, potential equivalent pairs are directed to either SAT or EPS, depending on the width of miter circuit formed by the pairs' TFI cones. Once the solving method is determined, the original CEC problem will be divided into subproblems, which are then solved in parallel by threads within a thread pool. However, uneven distribution of subproblems across threads can lead to load imbalances. To prevent this issue, PDP-CEC incorporates a dynamic task scheduling workflow, including a binary splitting tree to guide task scheduling.

The workflow of PDP-CEC is illustrated in Fig. 4. Benefiting from the independence of the subproblems generated by partitioning, the framework adopts a "master-worker" structure. Avoiding complexity in control logic, we designed three types of threads: the master thread, the monitor thread, and worker threads in the thread pool.

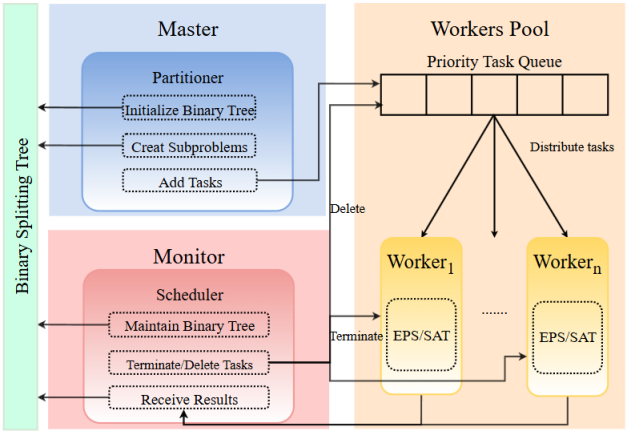

Fig. 4. The Workflow of PDP-CEC.

Master thread initializes the binary splitting tree with the help of the partitioner. In situations where the task queue has gaps, it identifies unresolved leaf nodes, generates new subproblems via the partitioner, adds these to the task queue, and facilitates the growth of the binary splitting tree.

Monitor thread serves as a central control mechanism for managing task scheduling within the workflow. It works alongside the master thread to oversee the node information stored in the binary splitting tree, controls the states of tasks, propagates information about nodes throughout the tree, and removes UNSAT node tasks from the task queue or instructs the threads currently handling those tasks to terminate.

Worker threads solve subproblems in the task queue concurrently, utilizing the solver designated for the original problem. A priority queue within the thread pool ensures that subproblems of lower complexity are solved first, with results being relayed to monitor thread for timely updates to the node information in the binary splitting tree.

A. Binary Splitting Tree

The binary splitting tree is primarily updated when Monitor thread receives new result information, which involves two main operations:

- **Inference towards the root node:** If a node is determined to be UNSAT, and its sibling node has also been proven UNSAT, then their UNSAT results can infer that their parent node is UNSAT. This process proceeds up towards the root node.
- **Propagation towards the leaf nodes:** If a node is UNSAT, then all its descendant nodes in that branch are also UNSAT.

B. Dynamic Task Scheduling Strategy

In the parallel process, each node in the tree has four states: unsubmitted, waiting, running, and completed. Initially, all nodes are in the "unsubmitted" state. Master thread sorts the leaf nodes according to (1), then adds them to the task queue where node's state is "waiting". Tasks in the queue are assigned to Worker threads, which update the nodes' states to

"running" before solving. After obtaining the results, Monitor thread retrieves the task IDs and updates the corresponding node's state to "completed".

We present an execution example in Fig. 5 that demonstrates our scheduling method. Five Worker threads are used, and the left side of the figure shows the binary splitting tree at an intermediate stage. After node 6 is completed and its UNSAT result is passed to Monitor thread, it infers node 3 as UNSAT since both nodes 6 and 7 are UNSAT. Node 2, the sibling of node 3, is subsequently pushed into the task queue and picked up by an idle thread. The propagation from node 6 triggers the termination of the corresponding worker for node 10 and marks node 11 as UNSAT. Since there are still idle threads available, Master thread identifies the next unresolved leaf node, node 8, splits it, and appends nodes 12 and 13 to the tree. Both nodes 12 and 13 are then queued for execution, with the idle Worker thread taking on node 12, while node 13 remains in the task queue.

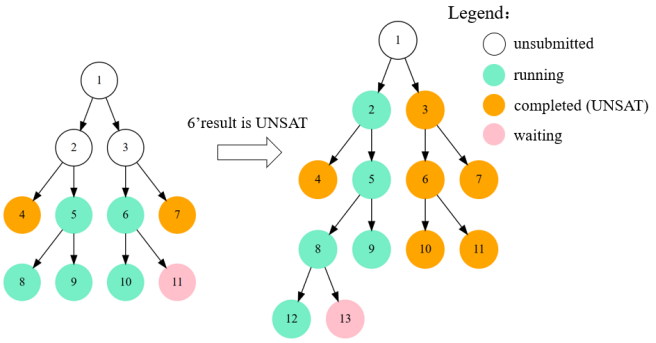

Fig. 5. A running example of our dynamic task scheduling strategy.

V. Experimental Results

In this section, we present the experimental setup and compare PDP-CEC with current representative CEC tools, such as ABC and Hybrid-CEC. Then we identify the factors contributing to the superior performance of PDP-CEC compared to Hybrid-CEC. Finally, we analyze the scalability of PDP-CEC.

A. Experimental Setup

All experiments were performed on a server with 128 Intel Xeon Platinum 8153 cores (2.00 GHz) and 2048 GB of memory, running CentOS 7. PDP-CEC is implemented in C++ and compiled with GNU g++ (version 9.3.1), with all competing methods compiled and executed under the same conditions. Our implementation utilizes the Metis [20] library for graph partitioning and the standard library for parallelism.

The benchmarks used in our experiments are derived from the Integrated Circuit EDA Elite Challenge 2022 [21] with modifications. We select 24 datapath circuits, featuring various sizes and high widths (higher than 32), including multipliers and adders. Each of the 24 instances consists of two functionally equivalent circuits in AIG format and has only a single PO, making parallel method [13] that depends on TFI unsuitable for these benchmarks.

For each experiment, the running time limit for each instance is set to 7200 seconds, and 'TO' means timeout. The unit of running time is seconds, and experiments were conducted with 128 threads by default.

B. Assessment of Datapath Benchmarks

We compare the performance of PDP-CEC with three other CEC tools in Table I:

- OPCEC: A simplified PDP-CEC version without pre-partitioning or the monitor's task submission strategy.
- ABC-SP: The split-based parallel CEC command in ABC, using "&splitprove -P 100 -L 7" for comparison.
- H-CEC: The Hybrid-CEC, a serial solution for datapath CEC.

TABLE I
Performance Comparison of Different CEC Methods on Datapath Benchmarks

Instance	Gates	PDP-CEC	OPCEC	ABC-SP	H-CEC
dpath_1	735	**9.63**	10.54	120.35	200.56
dpath_2	1034	**417.44**	523.08	1415.43	TO
dpath_3	1037	1386.46	**1172.67**	1377.82	TO
dpath_4	6905	**7.57**	8.97	TO	265.01
dpath_5	8003	**2771.45**	TO	TO	TO
dpath_6	6218	**7.21**	8.65	TO	298.92
dpath_7	7073	**12.39**	18.33	TO	1552.04
dpath_8	7994	**2917.33**	TO	TO	TO
dpath_9	6227	**7.25**	8.30	TO	156.46
dpath_10	7076	**14.32**	17.94	TO	490.71
dpath_11	7997	**4162.91**	TO	TO	TO
dpath_12	7079	**15.04**	17.86	TO	585.54
dpath_13	8000	**3180.89**	TO	TO	TO
dpath_14	8063	**4138.11**	TO	TO	TO
dpath_15	8087	**4158.53**	TO	TO	TO
dpath_16	8078	**4940.52**	TO	TO	TO
dpath_17	8066	**3926.71**	TO	TO	TO
dpath_18	8075	**4774.12**	TO	TO	TO
dpath_19	8072	**4134.73**	TO	TO	TO
dpath_20	8081	**4863.37**	TO	TO	TO
dpath_21	8084	**5108.63**	TO	TO	TO
dpath_22	8069	**3891.60**	TO	TO	TO
dpath_23	8090	**3788.22**	TO	TO	TO
dpath_24	9134	TO	TO	TO	TO
Best		22	1	0	0
Solved		23	9	3	7

The comparison results in Table I show that PDP-CEC is consistently the fastest among all methods. Compared to H-CEC, PDP-CEC solves 23 out of 24 instances within 7200 seconds, about 3x more than H-CEC. For instances completing within the time limit, PDP-CEC achieves speedups ranging from 5.11x to 125.27x over H-CEC, with the greatest benefits observed in larger circuits.

PDP-CEC also outperforms ABC-SP, whose search-space splitting strategy resembles PDP-CEC's. However, with the *NodeSelecting* algorithm, PDP-CEC generates more efficient subproblems, while its dynamic task scheduling strategy enables faster subproblem resolution, which results in more solved instances and reduced running time compared to ABC-SP. Finally, PDP-CEC outperforms OPCEC on all other in-

stances except dpath_3, where OPCEC performs better. This is because the key subproblems for dpath_3 are located in the lower levels of the binary splitting tree, a common occurrence in smaller circuits. Overall, the results highlight PDP-CEC's advantage due to its effective task scheduling and partitioning approach.

C. Comparative Analysis of PDP-CEC and Hybrid-CEC

TABLE II
PERFORMANCE COMPARISON OF SAT AND EPS BETWEEN H-CEC AND PDP-CEC WITH 128 THREADS.

Instance	Gates	T_{SAT}		T_{EPS}	
		PDP-CEC	H-CEC	PDP-CEC	H-CEC
dpath_14	8063	4120.48	TO	9.33	331.04
dpath_15	8087	4138.71	TO	9.82	339.92
dpath_16	8078	4939.53	TO	10.99	836.79
dpath_17	8066	3906.93	TO	9.78	423.97
dpath_18	8075	4754.60	TO	9.52	452.88
dpath_19	8072	4113.94	TO	10.79	1685.97
dpath_20	8081	4843.47	TO	9.90	1570.13
dpath_21	8084	5088.58	TO	10.05	1369.43
dpath_22	8069	3871.51	TO	10.09	355.38
dpath_23	8090	3768.02	TO	10.20	454.33

PDP-CEC also makes acceleration on Exact Probability-based Simulation (EPS), which is the key method in H-CEC. Table II shows the performance comparison between PDP-CEC and H-CEC on SAT and EPS. In this experiment, we extract 10 instances from the benchmark set, which H-CEC cannot solve. These results highlight that H-CEC struggles to handle these instances because of its insufficient SAT component. In contrast, PDP-CEC successfully resolves pairs identified as intractable SAT problems, which is critical for solving more complex datapath circuits. Additionally, PDP-CEC achieves speedups ranging from 35.48x to 158.60x over H-CEC for EPS. While the speedup is expected to be linear with the number of threads, slight deviations occur as a result of the overhead associated with the thread pool and graph partitioning. Remarkably, superlinear speedups are possible with sufficient system support.

D. Scalability Analysis of PDP-CEC

TABLE III
EXECUTION TIME WITH VARIOUS THREADS FOR PDP-CEC.

Instance	PDP-CEC				
	8t	16t	32t	64t	128t
dpath_1	33.62	26.75	16.38	13.74	9.63
dpath_4	16.49	8.52	7.25	8.30	7.57
dpath_6	14.49	8.67	9.92	8.69	7.21
dpath_7	48.59	31.23	20.84	19.37	12.39
dpath_9	19.03	11.12	9.45	8.29	7.25
dpath_10	55.20	31.64	19.47	18.92	14.32
dpath_12	56.26	30.20	22.30	19.94	15.04
average	34.81	21.16	15.09	13.89	10.49

We assess the scalability of PDP-CEC by varying the thread count from 8 to 128. The instances selected for this experiment, listed in Table III, were solved by PDP-CEC across different thread configurations. To minimize estimation errors caused by the extended running time of instances dpath_2, we report the average running time of other instances in Fig. 6, with dpath_2 shown separately as a distinct line. At lower thread counts, both average line and dpath_2 line show a near-linear speedup. However, the running time converges more gradually as threads increase, due to additional overhead from thread management and graph partitioning. In summary, the results showcase PDP-CEC as a scalable solution with strong potential for adaptation to distributed systems, supporting application to even larger datapath circuits.

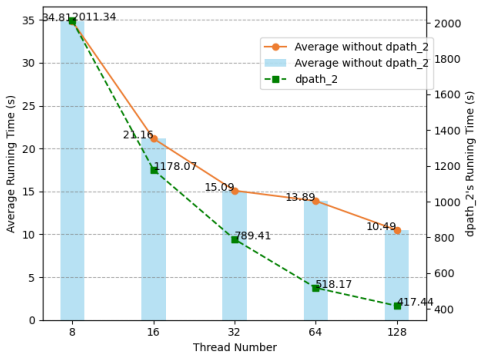

Fig. 6. The line-bar chart of execution time with various threads for PDP-CEC. Average data is come from Table III. The instance dpath_2 is separately illustrated.

VI. Conclusion

This study introduces a novel Parallel Dynamic Partitioning CEC approach, PDP-CEC, breaking the limitations of existing CEC methods, particularly in solving datapath CEC problems. By exploiting the inherent properties of circuits, PDP-CEC employs search space splitting in circuit partitioning for datapath CEC problem. An algorithm is designed in PDP-CEC for selecting assigned nodes and generate massive subproblems efficiently without dependencies. PDP-CEC applies a dynamic task scheduling strategy to manage the parallel execution of subproblems, preventing stalled workers and ensuring efficient resource utilization. Compared to three other CEC tools, PDP-CEC demonstrates superior performance, solving more instances and reducing overall running time. In conclusion, PDP-CEC demonstrates strong capability in solving datapath CEC problems, providing a scalable and efficient solution that surpasses the performance limitations of existing methods.

Future research will aim to extend PDP-CEC to distributed systems, further enhancing scalability and enabling applications on larger datapath circuit designs.

Acknowledgment

This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDA0320000 and XDA0320300 and the Postdoctoral Fellowship Program (Grade B) of China Postdoctoral Science Foundation, Grant No. GZB20240760.

REFERENCES

[1] M. Fujita, "Toward unification of synthesis and verification in topologically constrained logic design," *Proc. IEEE*, vol. 103, no. 11, pp. 2052–2060, Nov. 2015.

[2] Z. Chu, M. Soeken, Y. Xia, L. Wang, and G. De Micheli, "Structural rewriting in XOR-majority graphs," in *Proc. 24th Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2019, pp. 663–668.

[3] Brayton, Robert K., Gary D. Hachtel, and Alberto L. Sangiovanni-Vincentelli. "Multilevel logic synthesis." Proceedings of the IEEE 78.2 (1990): 264-300.

[4] L. Amarú, F. Marranghello, E. Testa, C. Casares, V. Possani, J. Luo, P. Vuillod, A. Mishchenko, and G. De Micheli, "SAT-sweeping enhanced for logic synthesis," in *Proc. 57th ACM/IEEE Design Automation Conf. (DAC)*, 2020, pp. 1–6.

[5] S. Chatterjee, A. Mishchenko, R. K. Brayton, X. Wang, and T. Kam, "Reducing structural bias in technology mapping," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 12, pp. 2894–2903, Dec. 2006.

[6] A. Mishchenko, M. Case, R. Brayton, and S. Jang, "Scalable and scalably-verifiable sequential synthesis," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2008, pp. 234–241.

[7] V. Sklyarov, I. Skliarova, and A. Kabulov, "Hardware accelerators for solving computationally intensive problems over binary vectors and matrices," in *AIP Conf. Proc.*, vol. 2781, no. 1, 2023.

[8] S. Harris and D. Harris, *Digital Design and Computer Architecture*, 2015. Morgan Kaufmann.

[9] A. Mishchenko, S. Chatterjee, R. Brayton, and N. Een, "Improvements to combinational equivalence checking," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2006, pp. 836–843.

[10] A. Kuehlmann, "Dynamic transition relation simplification for bounded property checking," in *IEEE/ACM Int. Conf. on Computer Aided Design, 2004. ICCAD-2004.*, pp. 50–57, 2004.

[11] A. Mishchenko, "ABC: A system for sequential synthesis and verification," *URL http://www.eecs.berkeley.edu/alanmi/abc*, vol. 17, 2007.

[12] A. Mishchenko, S. Chatterjee, R. Jiang, and R. K. Brayton, "FRAIGs: A unifying representation for logic synthesis and verification," ERL Tech. Rep., 2005.

[13] V. N. Possani, A. Mishchenko, R. P. Ribas, and A. I. Reis, "Parallel combinational equivalence checking," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 3081–3092, Oct. 2019.

[14] H. Wang and S. Beamer, "RepCut: Superlinear parallel RTL simulation with replication-aided partitioning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 572–585.

[15] M. Zhao, S. Cai, and Y. Qian, "Distributed SMT Solving Based on Dynamic Variable-Level Partitioning," in *International Conference on Computer Aided Verification*, 2024, pp. 68–88.

[16] G. Karypis and V. Kumar, "Multilevel k-way partitioning scheme for irregular graphs," *Journal of Parallel and Distributed Computing*, vol. 48, no. 1, pp. 96–129, 1998.

[17] P. Bjesse and A. Boralv, "DAG-aware circuit compression for formal verification," in *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004.*, pp. 42–49, 2004: IEEE.

[18] Z. Chen, X. Zhang, Y. Qian, Q. Xu, and S. Cai, "Integrating exact simulation into sweeping for datapath combinational equivalence checking," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2023, pp. 1–9.

[19] S.-C. Wu, C.-Y. Wang, and J.-A. Hsieh, "The potential and limitation of probability-based combinational equivalence checking," in *2006 15th Asian Test Symposium*, pp. 103–108, 2006: IEEE.

[20] G. Karypis, "Metis: A Graph Partitioning Library," 2024. [Online]. Available: https://github.com/KarypisLab/METIS. [Accessed: Oct. 9, 2024].

[21] ICISC, "ICISC EDA Challenge," 2022. [Online]. Available: https://eda.icisc.cn/. [Accessed: Oct. 9, 2024].